Embedded Linux and Real-Time Operating System Development for Internet of Things Applications

Youssef Khaled Omar Mahmoud
Department of Electrical, Electronic and Communication Engineering
Faculty of Engineering, Cairo University
Giza, Egypt
youefkh05@gmail.com
ORCID: 0009-0005-5807-4381

***Abstract*—This paper presents the work completed during the IoT Summer Research Internship 2026 at the Nanoelectronics Integrated Systems Center (NISC), Nile University. The internship focused on embedded systems, Internet of Things (IoT), and real-time operating systems through a series of hands-on tasks. The developed solutions demonstrated communication between embedded devices and cloud services while providing practical experience in embedded software development, cloud technologies, wireless communication, and system integration.**

***Index Terms*—Internet of Things (IoT), Embedded Systems, Raspberry Pi, Embedded Linux, TI-RTOS, CC1352P1 LaunchPad, MQTT, HTTP, InfluxDB, Grafana, UART, Bluetooth, Zigbee.**

I. Introduction

The Nile University Summer Research Internship 2026 provided an opportunity to gain practical experience in embedded systems, Internet of Things (IoT), and real-time software development through a series of hands-on engineering tasks. The internship emphasized applying theoretical knowledge to real-world embedded applications while introducing modern software and communication technologies widely used in industry.

Throughout the internship, three major technical areas were explored. The first focused on Embedded Linux development using the Raspberry Pi, where Python was employed to interface with hardware peripherals and implement different GPIO control techniques. The second addressed the design of an end-to-end IoT system by transmitting sensor measurements from edge devices to cloud services using both HTTP and MQTT communication protocols. The collected data were stored in a time-series database and visualized through a real-time monitoring dashboard. Finally, the internship introduced real-time embedded software development using TI-RTOS on the Texas Instruments CC1352P1 LaunchPad, where multiple concurrent tasks including LED control, Bluetooth communication, Zigbee communication, and UART-based communication with the Raspberry Pi were implemented.

In addition to software implementation, the internship emphasized understanding the design trade-offs between different communication protocols and software architectures. The progression from basic hardware interfacing to cloud connectivity and finally to multitasking embedded systems provided a comprehensive view of modern IoT system development.

This report summarizes the assigned tasks, describes the methodologies followed during implementation, presents the achieved results, and discusses the technical knowledge and practical skills acquired throughout the internship.

II. Assigned Tasks

The internship activities were organized progressively over several weeks, with each stage building upon the knowledge and skills acquired in the previous one. [1]

Week 1: Embedded Linux and Raspberry Pi

- Raspberry Pi development using Embedded Linux.
- Python programming for hardware interfacing.
- GPIO programming and sensor interfacing.

Weeks 2–3: Internet of Things (IoT)

- Cloud communication using HTTP.
- Cloud communication using MQTT.
- Data storage using InfluxDB.
- Data visualization using Grafana.

Weeks 3–4: Real-Time Operating Systems (RTOS)

- Real-Time Operating System (RTOS) programming.
- Bluetooth communication.
- Zigbee communication.
- UART-over-USB communication.
- Integration between the RTOS microcontroller and the Raspberry Pi.

III. Brief Description of Assigned Tasks

The internship was organized into three major technical tasks that progressively introduced embedded systems, Internet of Things (IoT), and real-time operating systems (RTOS). Each task combined theoretical concepts with practical implementation, allowing the development of complete embedded applications and their integration into an end-to-end IoT system. [1]

A. Raspberry Pi Development

1) Objective: The objective of this task was to develop a solid foundation in software development on Embedded Linux while learning the fundamentals of hardware interfacing using the Raspberry Pi. The work progressed from C programming exercises to GPIO-based control applications, providing practical experience in software design, Linux development, and interaction with external hardware peripherals.

2) Task 2.1 – C Programming Fundamentals: The first stage focused on strengthening programming skills using the C language within the Raspberry Pi Linux environment. The implemented application solved the Emirp Number problem, where prime numbers whose digit-reversed values are also prime were identified within a user-defined range.

The program was developed using a modular design by separating the functionality into helper functions responsible for primality testing and digit reversal. Dynamic memory allocation was employed to store the resulting Emirp pairs, while structures were used to organize the associated data, including the prime number, its inverse, and the numerical difference between them. Input validation was also incorporated to detect invalid ranges and incorrect user entries before executing the algorithm.

This task provided practical experience with:

- Programming in C under Embedded Linux.
- Modular software development using functions.
- Structure-based data organization.
- Dynamic memory allocation using `malloc()`.
- Input validation and error handling.
- Compilation and execution using the GNU Compiler Collection (GCC).

The overall workflow of the developed Emirp application is illustrated in Figure 1. The program begins by validating the user inputs, allocates memory dynamically for storing valid Emirp pairs, iterates through the specified range to identify qualifying numbers, and finally computes statistical information before displaying the results. Example executions are presented in Figure 2, while representative input-output cases are summarized in Table I.

TABLE I: Representative execution scenarios of the Emirp number application. [1]

Input Range	Observed Result
11 – 100	Emirp pairs successfully identified and summarized.
101 – 500	Larger set of Emirp numbers processed correctly.
50 – 10	Invalid range detected and an appropriate error message displayed.
Negative or even inputs	Input validation rejected invalid values before execution.

Table I summarizes representative execution scenarios of the developed application.

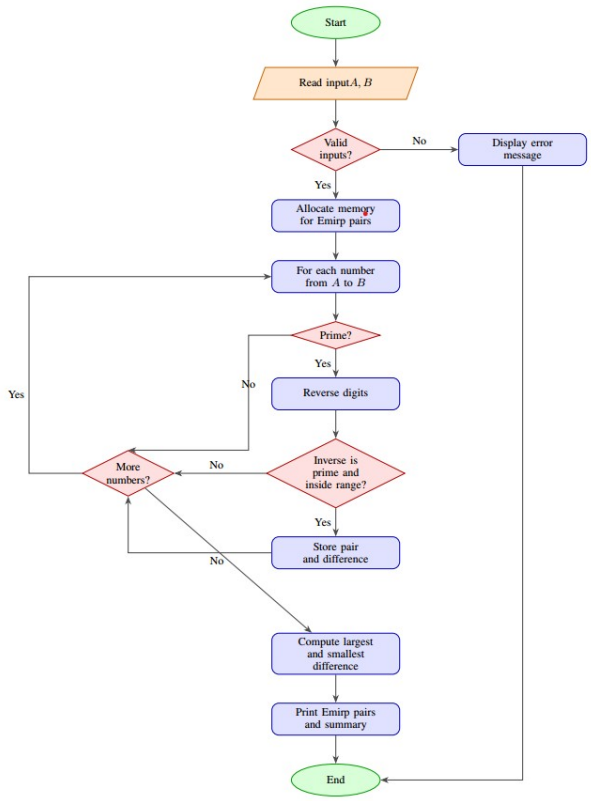
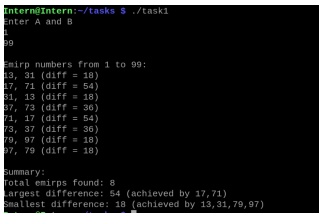
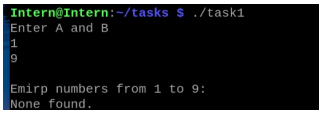


Fig. 1: Flowchart of the implemented Emirp number detection algorithm. [1]

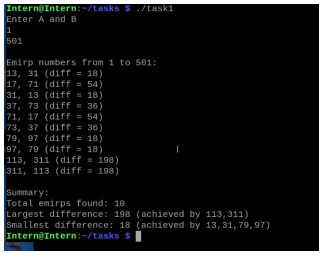
While Figure 2 presents sample terminal outputs obtained during testing.



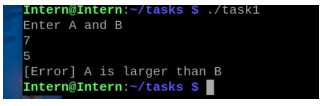
(a) Typical execution.



(b) Small range.



(c) Large range.



(d) Invalid input.

Fig. 2: Terminal outputs demonstrating successful execution and input validation of the Emirp number application. [1]

3) Task 2.2 – Embedded Linux and GPIO Programming: The second stage introduced hardware interfacing using the Raspberry Pi GPIO pins. Python was used to control LEDs

and read the state of a push button through the GPIO interface provided by the Linux operating system.

Three different software approaches were implemented for LED control:

- **Delay-based control**, where software delay loops generated the blinking period.
- **Timer-based control**, where timing mechanisms were used to improve execution accuracy.
- **Interrupt-based control**, where GPIO interrupts enabled immediate response to button presses without continuous polling.

These implementations demonstrated how different software architectures influence processor utilization, timing accuracy, and system responsiveness while using identical hardware.

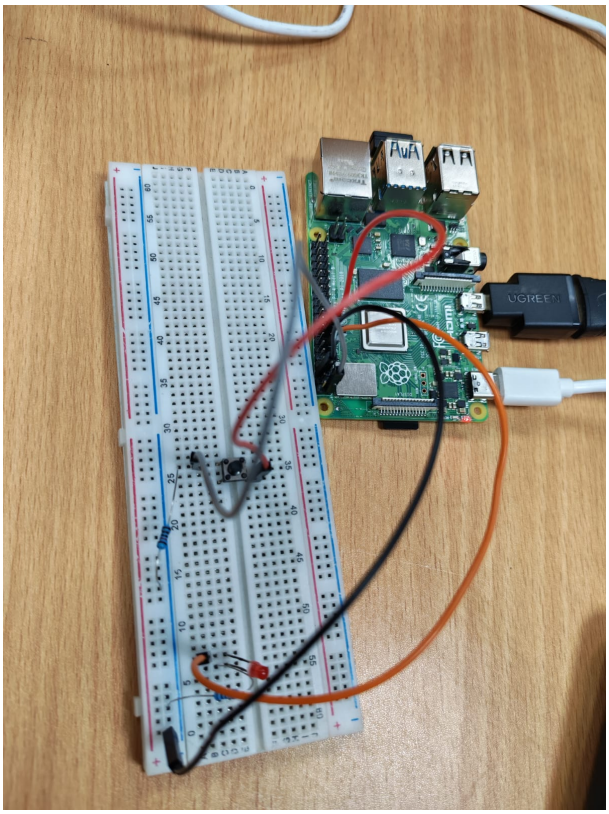


Fig. 3: GPIO-based LED control implemented on the Raspberry Pi. [2]

4) Outcomes: Example executions of the developed C application are presented in Figure 2, while Figure 3 illustrates the GPIO-based LED control implemented on the Raspberry Pi.

The Raspberry Pi tasks successfully demonstrated both software development and embedded hardware interfacing. The C programming exercise reinforced fundamental programming concepts, including modular design, dynamic memory management, and algorithm implementation. The GPIO applications provided practical experience in controlling external peripherals, implementing event-driven programming techniques, and comparing different software control methodologies. Figure 3 illustrates the implemented GPIO-based LED control setup.

B. Internet of Things (IoT)

1) Objective: The objective of this task was to develop a complete Internet of Things (IoT) data acquisition system capable of collecting sensor measurements, transmitting them to cloud platforms, storing them in a time-series database, and visualizing the acquired data in real time. Throughout the internship, two communication approaches were investigated. The initial implementation utilized HTTP requests with the Adafruit IO platform, while the final system adopted the MQTT publish/subscribe protocol together with InfluxDB and Grafana to improve communication efficiency and enable scalable real-time monitoring.

2) Work Performed: The implemented IoT system consisted of multiple software components working together to provide end-to-end data acquisition and visualization. Sensor measurements were collected by the Raspberry Pi using Python, after which the measured values were transmitted to cloud services. During the first implementation, the Raspberry Pi periodically uploaded sensor readings to Adafruit IO using the HTTP protocol. Although functional, this approach required continuous polling and repeated request generation.

The second implementation migrated the communication layer to MQTT. The Raspberry Pi published sensor measurements to an MQTT broker whenever new data became available. The broker forwarded the received messages to InfluxDB for permanent storage, while Grafana retrieved the stored measurements directly from the database to generate live dashboards and historical plots.

Figure 4 illustrates the complete IoT architecture implemented during the internship.

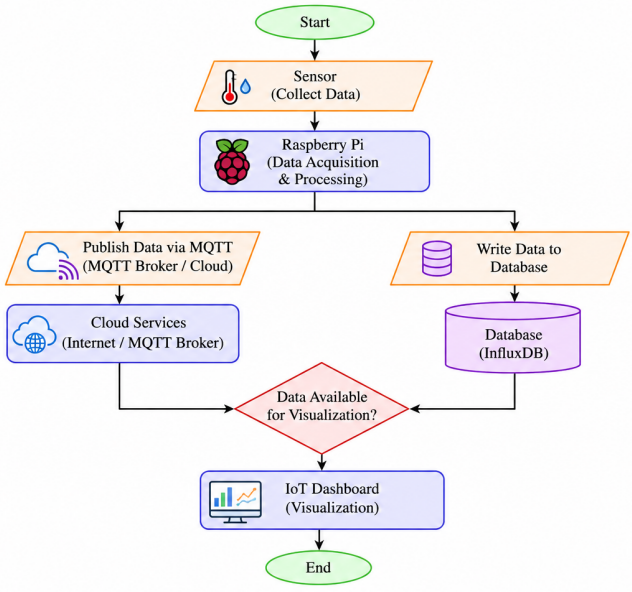
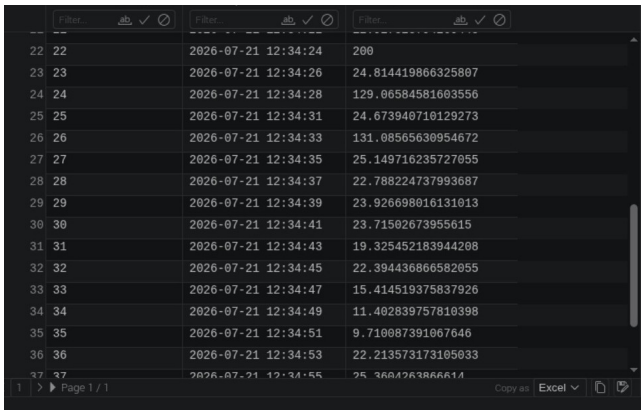


Fig. 4: Overall IoT architecture implemented during the internship. [1]

3) Database Integration: To enable persistent storage and historical analysis of sensor measurements, the received MQTT messages were stored in an InfluxDB time-series database. Unlike conventional relational databases, InfluxDB is specifically designed for timestamped data, making it well suited for Internet of Things (IoT) applications where measurements are continuously generated over time. Each sensor reading was recorded together with its corresponding timestamp, allowing efficient querying, filtering, and long-term storage of collected data. This database served as the primary data source for the Grafana visualization platform, enabling both live monitoring and analysis of previously recorded measurements.

Figure 5 shows a sample of the stored measurements within the InfluxDB database.



22	22	2026-07-21 12:34:24	200
23	23	2026-07-21 12:34:26	24.814419866325807
24	24	2026-07-21 12:34:28	129.06584581603556
25	25	2026-07-21 12:34:31	24.673940710129273
26	26	2026-07-21 12:34:33	131.08565630954672
27	27	2026-07-21 12:34:35	25.149716235727055
28	28	2026-07-21 12:34:37	22.788224737993687
29	29	2026-07-21 12:34:39	23.926698016131013
30	30	2026-07-21 12:34:41	23.71502673955615
31	31	2026-07-21 12:34:43	19.325452183944208
32	32	2026-07-21 12:34:45	22.394436866582055
33	33	2026-07-21 12:34:47	15.414519375837926
34	34	2026-07-21 12:34:49	11.402839757810398
35	35	2026-07-21 12:34:51	9.710087391067646
36	36	2026-07-21 12:34:53	22.213573173105033

Fig. 5: Example of sensor measurements stored in the InfluxDB time-series database. [1]

4) Methodology: Two communication protocols were implemented and evaluated throughout the internship.

The first implementation employed the HTTP protocol, where the Raspberry Pi continuously transmitted sensor measurements to the cloud using periodic HTTP requests. Since the client must repeatedly request communication regardless of whether new data are available, this approach increases network traffic, processor activity, and power consumption.

The second implementation adopted the MQTT publish/subscribe communication model. Instead of continuously polling the server, the Raspberry Pi publishes sensor values only when new measurements become available. Subscribers automatically receive updates from the MQTT broker whenever new messages are published. Consequently, MQTT significantly reduces unnecessary network traffic while providing lower latency and improved scalability for IoT applications.

Table II summarizes the characteristics of both communication protocols.

TABLE II: Comparison between HTTP and MQTT communication protocols.

Feature	**HTTP**	**MQTT**
Communication Model	Request/Response	Publish/Subscribe
Data Transfer	Continuous polling	Only when new data are available
Bandwidth Usage	Higher	Lower
Power Consumption	Higher	Lower
Scalability	Moderate	High
Real-Time Updates	Limited by polling interval	Immediate event-driven updates
IoT Suitability	Good	Excellent

5) Outcomes: The developed IoT platform successfully demonstrated complete cloud connectivity and real-time monitoring capabilities. Sensor measurements were reliably transmitted from the Raspberry Pi to the MQTT broker, where published messages were successfully received and forwarded to the remaining components of the system. The acquired data were subsequently stored within an InfluxDB time-series database and visualized through interactive dashboards.

Figure 6 shows the MQTT broker successfully receiving and forwarding the published sensor measurements.

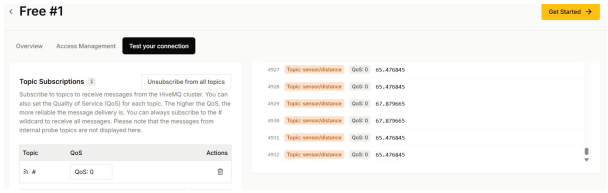
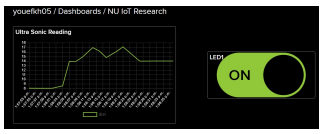


Fig. 6: MQTT broker receiving published sensor measurements. [2]

The transition from HTTP to MQTT significantly improved communication efficiency by eliminating continuous polling and reducing unnecessary network traffic. In addition, the integration of InfluxDB and Grafana enabled persistent storage, historical data analysis, and live visualization, providing a scalable architecture suitable for future IoT deployments.

Example dashboards generated during the internship are presented in Figure 7.



(a) Adafruit IO dashboard.

(b) Grafana dashboard.

Fig. 7: Cloud dashboards used for monitoring sensor measurements. [3], [4]

C. Real-Time Operating System (RTOS)

1) Objective: The objective of this task was to gain practical experience in real-time embedded software development by implementing concurrent applications using a Real-Time Operating System (RTOS). The work focused on understanding task management, scheduling mechanisms, inter-task execution, and communication between embedded devices while developing a multitasking application on a wireless microcontroller platform.

2) Work Performed: The RTOS application was organized around a central kernel responsible for scheduling multiple independent tasks. Each task executed concurrently while sharing the processor according to the scheduling policy provided by the operating system. This architecture allowed different application functionalities to execute independently without blocking one another.

The implemented software consisted of several concurrent tasks responsible for hardware control, wireless communication, and external device interaction. Synchronization between these tasks was managed by the RTOS scheduler, ensuring deterministic execution and efficient processor utilization.

Figure 8 illustrates the overall RTOS architecture implemented during the internship.

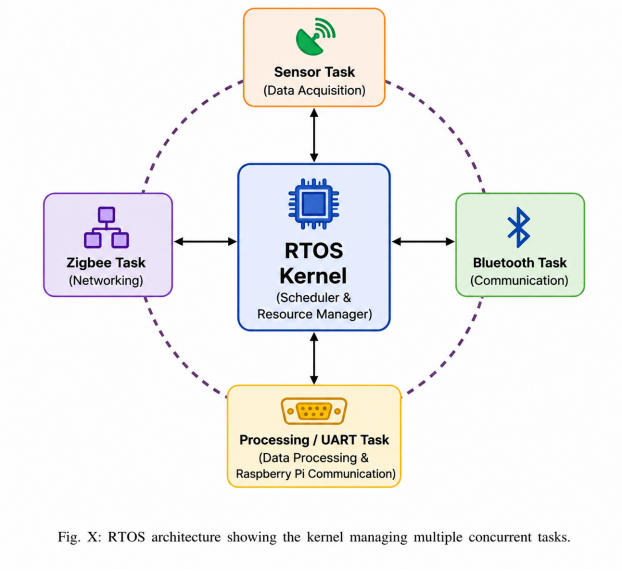


Fig. 8: RTOS architecture showing the kernel managing multiple concurrent tasks. [1]

3) Implemented Modules: Four independent application tasks were developed during this stage:

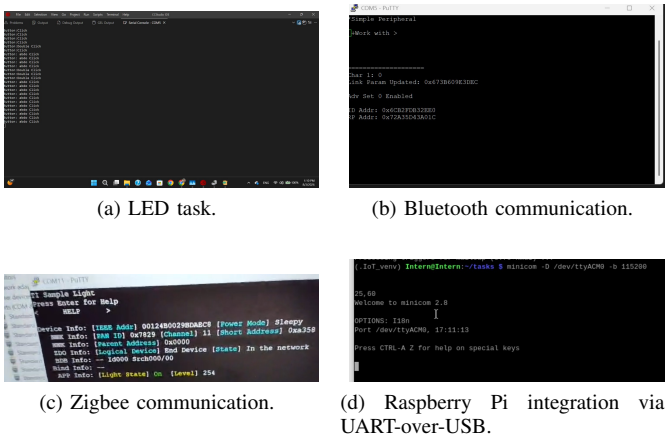
- **LED Task**: Periodically controlled the onboard LED to demonstrate periodic task scheduling.
- **Bluetooth Task**: Implemented Bluetooth communication for wireless data exchange.
- **Zigbee Task**: Established Zigbee communication for low-power wireless networking.
- **UART-over-USB Task**: Enabled serial communication between the RTOS device and the Raspberry Pi for system integration.

Each task executed independently under the supervision of the RTOS kernel, demonstrating concurrent execution without interfering with the operation of other tasks.

4) Outcomes: The RTOS implementation successfully demonstrated multitasking capabilities through the concurrent execution of independent application tasks. The scheduler efficiently managed processor time among the implemented tasks while maintaining responsive system behavior.

Successful Bluetooth and Zigbee communication verified the operation of multiple wireless interfaces under the RTOS environment. In addition, UART-over-USB communication established reliable integration between the RTOS microcontroller and the Raspberry Pi, allowing both platforms to exchange data as part of a unified embedded system.

Representative experimental results obtained during the implementation are presented in Figure 9.



(a) LED task.

(b) Bluetooth communication.

(c) Zigbee communication.

(d) Raspberry Pi integration via UART-over-USB.

Fig. 9: Experimental results obtained from the RTOS implementation. [2]

Acknowledgment

The author would like to express sincere gratitude to the Nile University Research Office for offering the opportunity to participate in the 2026 IoT Summer Research Internship Program.

Special thanks are extended to Prof. Ahmed Soltan, Director of the Nanoelectronics Integrated Systems Center (NISC), for providing this valuable opportunity, his continuous support, and for taking the time to listen to and discuss our feedback throughout the internship.

The author also gratefully acknowledges the guidance and supervision of Eng. Salma, Eng. Omar, and Eng. Ahmed, whose technical support, constructive feedback, and mentorship were invaluable throughout the internship.

References

[1] Y. K. Omar, "NU-IoT-Research-Intern," GitHub repository, 2026. Available: https://github.com/youefkh05/NU-IoT-Research-Intern

[2] Y. K. Omar, “NU IoT Research Internship Task Demonstrations,” Google Drive, 2026. Available: Google Drive Link
[3] Adafruit IO Dashboard, Available: https://io.adafruit.com/youefkh05/dashboards/nu-iot-research
[4] Grafana Public Dashboard, Available: https://broadpantry517.grafana.net/public-dashboards/eeee64a324cc4a5ea9bd651244ab6319